

Function Overloading

One Name, Multiple Missions

Compile-Time Polymorphism in C++

Lesson Title: Function Architectures

1. The Concept of Overloading

In programming, **Function Overloading** is the ability to create multiple functions with the exact same name, but with different input requirements. This is a form of Polymorphism (many forms).

The Problem: Without overloading, you would need names like `addInts(a, b)`, `addDoubles(a, b)`, and `addThreeInts(a, b, c)`. This clutters your brain and your code.

The Real-Life Analogy

Think of the word "**Open**":

- `Open(Door)`
- `Open(Bottle)`
- `Open(Account)`

The action "Open" is the same name, but the way you do it depends entirely on what you are opening. C++ functions work exactly the same way.

2. The Compiler's Decision

How does C++ know which function to run if they all have the same name? It looks at the **Function Signature**.

The Signature includes:

Number of parameters

Type of parameters

Order of parameter
types

When you call an overloaded function, the compiler compares the arguments you provided against the available signatures and performs a "best match" selection at compile time.

Crucial Rule: The **Return Type** (e.g., `int` vs `void`) is NOT part of the signature. You cannot overload functions by only changing what they return.

3. Variation by Data Type

The most common use of overloading is to perform the same logic on different types of numerical data.

```
int plusFunc(int x, int y) { return x + y; } double plusFunc(double x,  
double y) { return x + y; } int main() { cout << plusFunc(10, 20); //  
Calls int version cout << plusFunc(10.5, 20.3); // Calls double version }
```

Why this is better:

The user of your function only needs to remember one name: `plusFunc`. They don't have to worry about whether they are passing integers or decimals; the code "just works."

4. Variation by Parameter Count

Sometimes you need to perform a task with varying amounts of input. Overloading handles this gracefully.

```
// Adds TWO numbers int add(int a, int b) { return a + b; } // Adds THREE
numbers int add(int a, int b, int c) { return a + b + c; }
```

This allows your functions to be flexible. You can build a calculator that adds two numbers today, and easily extend it to add three, four, or five numbers just by adding more overloads.

5. Variation by Parameter Order

C++ can even distinguish between functions based on the sequence of the types. This is useful for formatting or logging functions.

```
void display(int a, string b) { cout << "ID: " << a << " Name: " << b; }  
void display(string a, int b) { cout << "Name: " << a << " ID: " << b; }
```

Usage:

```
display(101, "Alice"); // Matches version 1 display("Bob", 202); //  
Matches version 2
```

Even though both functions take an `int` and a `string`, the order makes them unique signatures in the eyes of the compiler.

6. What ISN'T Overloading?

It is important to recognize what the compiler will reject as an error. Two functions are NOT overloaded if they only differ in ways the compiler ignores.

1. Different Return Types Only

```
int getVal() { return 5; } double getVal() { return 5.0; } // ✗ ERROR!
```

The compiler cannot know which one you want if you just write `getVal()`; without assigning it.

2. Different Variable Names Only

```
void move(int x, int y); void move(int a, int b); // ✗ ERROR!
```

The variable names (`x, y` vs `a, b`) are irrelevant to the signature. To the compiler, both are just `move(int, int)`.

7. Why Professional Devs Use It

1. Improving Readability

Clean code should read like a story. Functions with consistent names are easier to follow.

2. Consistency

Standard libraries (like `cout`) use overloading. Whether you print a string, an `int`, or a `char`, you use `<<`. Imagine if you had to use `cout_int` or `cout_string`!

3. Maintainability

Changing the logic for a named action is easier when all variations are grouped together under one name.

8. Summary & Practice

Final Checklist:

- Same Name? 
- Different Parameters? 
- Return Types the same/different? (Doesn't matter) 

Code Challenge:

The Area Calculator:

1. Create a function `findArea(int side)` for a square.
2. Create an overloaded `findArea(int length, int width)` for a rectangle.
3. Create an overloaded `findArea(double radius)` for a circle ($\text{Area} = 3.14 * r * r$).

Module Complete: Function Overloading

Part of the C++ Polymorphism Series